



# Day-14

## Python Modules

# Python Modules

- What is a Module?
- **Python Module** is a file that contains built-in functions, classes, **its** and variables. There are many **Python modules**, each with its specific work.
- In this article, we will cover all about Python modules, such as How to create our own simple module, Import Python modules, From statements in Python, we can use the alias to rename the module, etc.

# Create a Python Module

- To create a Python module, write the desired code and save that in a file with .py extension. Let's understand it better with an example:
- Example:
- Let's create a simple calc.py in which we define two functions, one add and another subtract.

- # A simple module, calc.py
- def add(x, y):
  - return (x+y)
- def subtract(x, y):
  - return (x-y)

# Import module in Python

- We can import the functions, and classes defined in a module to another module using the import statement in some other Python source file.
- When the interpreter encounters an import statement, it imports the module if the module is present in the search path.

- For example, to import the module `calc.py`, we need to put the following command at the top of the script.
- Now, we are importing the **calc** that we created earlier to perform add operation.
- Create `module.py` file and write this code to import `calc.py` file code

```
# importing module calc.py
```

```
import calc
```

```
print(calc.add(10, 2))
```

## Example-2

- Save this code in a file named mymodule.py
- `def greeting(name):`
- `print("Hello, " + name)`

- Create file main.py write the following code
- import mymodule

```
mymodule.greeting("Hello Student")
```





- Let's look at the simple example to open a file named "file.txt" (stored in the same directory) in read mode and printing its content on the console.
- Example
- #opens the file file.txt in read mode
- `fileptr = open("file.txt","r")`
- 
- if fileptr:
- `print("file is opened successfully")`
- Output:
- `<class '_io.TextIOWrapper'>`
- file is opened successfully

- In the above code, we have passed filename as a first argument and opened file in read mode as we mentioned r as the second argument. The fileptr holds the file object and if the file is opened successfully, it will execute the print statement
- The close() method
- Once all the operations are done on the file, we must close it through our Python script using the **close()** method. Any unwritten information gets destroyed once the **close()** method is called on a file object.
- We can perform any operation on the file externally using the file system which is the currently opened in Python; hence it is good practice to close the file once all the operations are done.
- The syntax to use the **close()** method is given below

- Syntax
- `fileobject.close()`
- Consider the following example.
- `# opens the file file.txt in read mode`
- `fileptr = open("file.txt","r")`
- 
- `if fileptr:`
- `print("file is opened successfully")`
- 
- `#closes the opened file`
- `fileptr.close()`

- After closing the file, we cannot perform any operation in the file. The file needs to be properly closed. If any exception occurs while performing some operations in the file then the program terminates without closing the file.
- We should use the following method to overcome such type of problem.
- try:
  - `fileptr = open("file.txt")`
  - `# perform file operations`
- finally:
  - `fileptr.close()`

# The with statement

- The with statement was introduced in python 2.5. The with statement is useful in the case of manipulating the files. It is used in the scenario where a pair of statements is to be executed with a block of code in between.
- The syntax to open a file using with the statement is given below.
- `with open(<file name>, <access mode>) as <file-pointer>:`
- `#statement suite`
- The advantage of using with statement is that it provides the guarantee to close the file regardless of how the nested block exits.
- It is always suggestible to use the with statement in the case of files because, if the break, return, or exception occurs in the nested block of code then it automatically closes the file, we don't need to write the close() function. It doesn't let the file to corrupt.
- Consider the following example.

- Example
- with open("file.txt",'r') as f:
- content = f.read();
- print(content)

# Writing the file

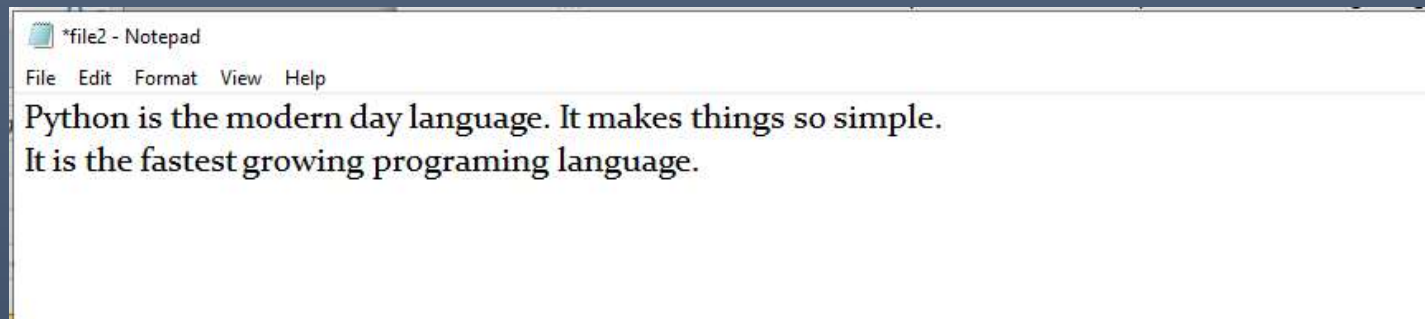
- To write some text to a file, we need to open the file using the open method with one of the following access modes.
- w: It will overwrite the file if any file exists. The file pointer is at the beginning of the file.
- a: It will append the existing file. The file pointer is at the end of the file. It creates a new file if no file exists.



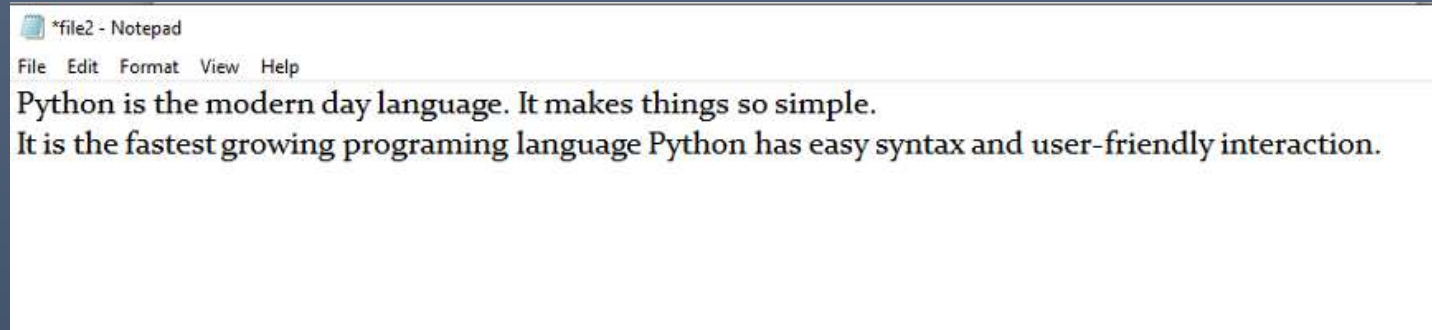
# Consider the following example.

- Example
- # open the file.txt in append mode. Create a new file if no such file exists.
- fileptr = open("file2.txt", "w")
- 
- # appending the content to the file
- fileptr.write("""Python is the modern day language. It makes things so simple.
- It is the fastest-growing programming language""")
- 
- # closing the opened the file
- fileptr.close()
- Output:
- File2.txt
- Python is the modern-day language. It makes things so simple. It is the fastest growing programming language.

- Snapshot of the file2.txt



- We have opened the file in w mode. The file1.txt file doesn't exist, it created a new file and we have written the content in the file using the write() function.
- Example 2
- #open the file.txt in write mode.
- fileptr = open("file2.txt","a")
- 
- #overwriting the content of the file
- fileptr.write(" Python has an easy syntax and user-friendly interaction.")
- 
- #closing the opened file
- fileptr.close()
- Output:
- Python is the modern day language. It makes things so simple.
- It is the fastest growing programming language Python has an easy syntax and user-friendly interaction.
- Output:
- Python is the modern day language. It makes things so simple.
- It is the fastest growing programming language Python has an easy syntax and user-friendly interaction.



- We can see that the content of the file is modified. We have opened the file in a mode and it appended the content in the existing file2.txt.
- To read a file using the Python script, the Python provides the read() method. The read() method reads a string from the file. It can read the data in the text as well as a binary format.
- The syntax of the read() method is given below.
- Syntax:
- `fileobj.read(<count>)`
- Here, the count is the number of bytes to be read from the file starting from the beginning of the file. If the count is not specified, then it may read the content of the file until the end.

# Consider the following example.

- Example
- #open the file.txt in read mode. causes error if no such file exists.
- `fileptr = open("file2.txt","r")`
- #stores all the data of the file into the variable content
- `content = fileptr.read(10)`
- # prints the type of the data stored in the file
- `print(type(content))`
- #prints the content of the file
- `print(content)`
- #closes the opened file
- `fileptr.close()`

- Output:
- `<class 'str'>`
- Python is
- In the above code, we have read the content of file2.txt by using the `read()` function. We have passed count value as ten which means it will read the first ten characters from the file.

If we use the following line, then it will print all content of the file.

- `content = fileptr.read()`
- `print(content)`
- Output:
- Python is the modern-day language. It makes things so simple.
- It is the fastest-growing programming language Python has easy an syntax and user-friendly interaction.



# Read file through for loop

- We can read the file using for loop. Consider the following example.
- #open the file.txt in read mode. causes an error if no such file exists.
- `fileptr = open("file2.txt","r");`
- #running a for loop
- `for i in fileptr:`
- `print(i)` # i contains each line of the file
- Output:
- Python is the modern day language.
- It makes things so simple.
- Python has easy syntax and user-friendly interaction.

# Read Lines of the file

- Python facilitates to read the file line by line by using a function `readline()` method. The `readline()` method reads the lines of the file from the beginning, i.e., if we use the `readline()` method two times, then we can get the first two lines of the file.
- Consider the following example which contains a function `readline()` that reads the first line of our file "file2.txt" containing three lines. Consider the following example.
- Example 1: Reading lines using `readline()` function
- `#open the file.txt in read mode. causes error if no such file exists.`
- `fileptr = open("file2.txt","r");`
- `#stores all the data of the file into the variable content`
- `content = fileptr.readline()`
- `content1 = fileptr.readline()`
- `#prints the content of the file`
- `print(content)`
- `print(content1)`
- `#closes the opened file`
- `fileptr.close()`

- Output:
- Python is the modern day language.
- It makes things so simple.
- We called the readline() function two times that's why it read two lines from the file.
- Python provides also the readlines() method which is used for the reading lines. It returns the list of the lines till the end of file(EOF) is reached.

## Example 2: Reading Lines Using readlines() function

- #open the file.txt in read mode. causes error if no such file exists.
- fileptr = open("file2.txt","r");
- 
- #stores all the data of the file into the variable content
- content = fileptr.readlines()
- 
- #prints the content of the file
- print(content)
- 
- #closes the opened file
- fileptr.close()

- Output:
- ['Python is the modern day language.\n', 'It makes things so simple.\n', 'Python has easy syntax and user-friendly interaction']

# Creating a new file

- The new file can be created by using one of the following access modes with the function `open()`.
- `x`: it creates a new file with the specified name. It causes an error a file exists with the same name.
- `a`: It creates a new file with the specified name if no such file exists. It appends the content to the file if the file already exists with the specified name.
- `w`: It creates a new file with the specified name if no such file exists. It overwrites the existing file.
- Consider the following example.

- Example 1
- #open the file.txt in read mode. causes error if no such file exists.
- `fileptr = open("file2.txt","x")`
- `print(fileptr)`
- if `fileptr`:
- `print("File created successfully")`
- Output:
- `<_io.TextIOWrapper name='file2.txt' mode='x' encoding='cp1252'>`
- File created successfully

# File Pointer positions

- Python provides the `tell()` method which is used to print the byte number at which the file pointer currently exists. Consider the following example.
- `# open the file file2.txt in read mode`
- `fileptr = open("file2.txt","r")`
- 
- `#initially the filepointer is at 0`
- `print("The filepointer is at byte :",fileptr.tell())`
- 
- `#reading the content of the file`
- `content = fileptr.read();`
- 
- `#after the read operation file pointer modifies. tell() returns the location of the fileptr.`
- 
- `print("After reading, the filepointer is at:",fileptr.tell())`



- Output:
- The filepointer is at byte : 0
- After reading, the filepointer is at: 117

# Modifying file pointer position

- In real-world applications, sometimes we need to change the file pointer location externally since we may need to read or write the content at various locations.
- For this purpose, the Python provides us the `seek()` method which enables us to modify the file pointer position externally.
- The syntax to use the `seek()` method is given below.
- Syntax:
- `<file-ptr>.seek(offset[, from])`

# The seek() method accepts two parameters:

- offset: It refers to the new position of the file pointer within the file.
- from: It indicates the reference position from where the bytes are to be moved. If it is set to 0, the beginning of the file is used as the reference position. If it is set to 1, the current position of the file pointer is used as the reference position. If it is set to 2, the end of the file pointer is used as the reference position.
- Consider the following example.

- Example
- # open the file file2.txt in read mode
- `fileptr = open("file2.txt","r")`
- 
- #initially the filepointer is at 0
- `print("The filepointer is at byte :",fileptr.tell())`
- 
- #changing the file pointer location to 10.
- `fileptr.seek(10);`
- 
- #tell() returns the location of the fileptr.
- `print("After reading, the filepointer is at:",fileptr.tell())`
- Output:
- 
- The filepointer is at byte : 0
- After reading, the filepointer is at: 10

# Python OS module

- Renaming the file
- The Python os module enables interaction with the operating system. The os module provides the functions that are involved in file processing operations like renaming, deleting, etc. It provides us the rename() method to rename the specified file to a new name. The syntax to use the rename() method is given below.
- Syntax:
  - rename(current-name, new-name)
  - The first argument is the current file name and the second argument is the modified name. We can change the file name bypassing these two arguments.

## Example 1:

- `import os`
  - 
  - `#rename file2.txt to file3.txt`
  - `os.rename("file2.txt","file3.txt")`
  - Output:
- 
- The above code renamed current file2.txt to file3.txt

# Removing the file

- The os module provides the remove() method which is used to remove the specified file. The syntax to use the remove() method is given below.
- remove(file-name)
- Example 1
- import os;
- #deleting the file named file3.txt
- os.remove("file3.txt")

# Creating the new directory

- The mkdir() method is used to create the directories in the current working directory. The syntax to create the new directory is given below.
- Syntax:
  - mkdir(directory name)
  - Example 1
- import os
- 
- #creating a new directory with the name new
- os.mkdir("new")



# The getcwd() method

- This method returns the current working directory.
- The syntax to use the getcwd() method is given below.
- Syntax
  - `os.getcwd()`
  - Example
    - `import os`
    - `os.getcwd()`
    - Output:
      - `'C:\\Users\\DEVANSH SHARMA'`

# Changing the current working directory

- The `chdir()` method is used to change the current working directory to a specified directory.
- The syntax to use the `chdir()` method is given below.
- Syntax
- `chdir("new-directory")`
- Example
- `import os`
- `# Changing current directory with the new directory`
- `os.chdir("C:\\Users\\DEVANSH SHARMA\\Documents")`
- `#It will display the current working directory`
- `os.getcwd()`
- Output:
- `'C:\\Users\\DEVANSH SHARMA\\Documents'`

# Deleting directory

- The `rmdir()` method is used to delete the specified directory.
- The syntax to use the `rmdir()` method is given below.
- Syntax
  - `os.rmdir(directory name)`
  - Example 1
    - `import os`
    - `#removing the new directory`
    - `os.rmdir("directory_name")`
    - It will remove the specified directory.

# Writing Python output to the files

- In Python, there are the requirements to write the output of a Python script to a file.
- The `check_call()` method of module `subprocess` is used to execute a Python script and write the output of that script to a file.
- The following example contains two python scripts. The script `file1.py` executes the script `file.py` and writes its output to the text file `output.txt`.
- Example
- `file.py`

- `temperatures=[10,-20,-289,100]`
- `def c_to_f(c):`
- `if c< -273.15:`
- `return "That temperature doesn't make sense!"`
- `else:`
- `f=c*9/5+32`
- `return f`
- `for t in temperatures:`
- `print(c_to_f(t))`
- `file.py`
  
- `import subprocess`
- 
- `with open("output.txt", "wb") as f:`
- `subprocess.check_call(["python", "file.py"], stdout=f)`

# Day Tasks

- **Exercise 1: Read text file into a variable and replace all newlines with space**
- **Given:** Assume you have a following text file (sample.txt).
- Contents in file
- Line1
- line2
- line3
- line4
- Line5
- Expected Output
- Line1 line2 line3 line4 line5

## Exercise 2

- Write a new file demo.txt & write the given contents in the file using python code after reading file it should display the file contents as a output
- “Hello welcome to all in Domain Computer”

## Exercise 3

- Open the file "demo.txt" and append content to the file:
- This is modified content of the file
- Exercise-4
- Delete the demo.txt file using python code



This PPT contents are

TO

© copyright  
all rights reserved

**Domain**®  
Computer Education  
TRAINING & PLACEMENT CENTRE



**TRAINING & PLACEMENT CENTRE**

**DOMAIN COMPUTER EDUCATION**

2nd floor, Balaji Celebration, Ganpati  
Mandir Road, Vishrambagh, Sangli

7620575756



**DOMAIN COMPUTER EDUCATION**

3rd Floor Prabhakar Plaza, Station road  
Kolhapur.

Ph.No.- 9225804090



Website: [www.domaincomputer.org](http://www.domaincomputer.org)